

Parallel Implementation of a Sequential Markov Chain in Monte Carlo Simulations of Physical Systems with Pairwise Interactions

Szymon Migacz,[†] Kajetan Dutka,[†] Przemysław Gumienny,[†] Maciej Marchwiany,[†] Dominik Gront,^{*,‡} and Witold R. Rudnicki^{*,§,†}

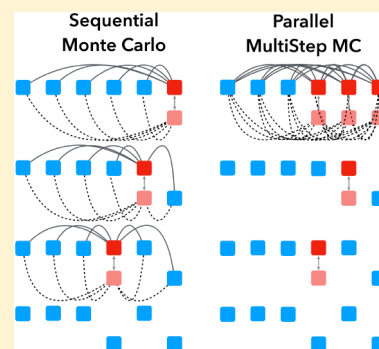
[†]Interdisciplinary Centre for Mathematical and Computational Modelling, University of Warsaw, Warsaw, Poland

[‡]Department of Chemistry, University of Warsaw, Warsaw, Poland

[§]Institute of Informatics, University of Białystok, Białystok, Poland

S Supporting Information

ABSTRACT: In molecular simulations performed by Markov Chain Monte Carlo (typically employing the Metropolis criterion), each state of a system is obtained by a small random modification of the previous state. Therefore, the process consists of an immense number of small, quick to calculate steps, which are inherently sequential and hence considered to be very hard to parallelise. Here, we present a novel protocol for efficient calculation of multiple sequential steps in parallel. To this end, we first precompute in parallel energy components of all states achievable in a sequence of steps. Then we select a single path through all achievable states, which is identical with the path obtained with the sequential algorithm. As an example, we carried out simulations of the TIP5P water model with the new protocol and compared results with those obtained using the standard Metropolis Monte Carlo scheme. The implementation on the Titan X (Pascal) graphic processor (GPU) architectures allows for a 30-fold speedup in comparison with a simulation on a single core of a multicore CPU. The protocol is general and not limited to the GPU; it can also be used on multicore CPU when the longest possible length of the single simulation is required.



1. INTRODUCTION

The Monte Carlo method^{1,2} was the first approach used for studying complex phenomena by the means of large-scale computer simulations. In particular, the application of the Metropolis protocol³ for establishing equilibrium properties in a canonical ensemble has become a standard method for studying physical systems of varying complexity starting from spin glasses⁴ through polymers to proteins and other biomolecules, in both a coarse-grained⁵ and atomistic representation.⁶

The alternative method for studying atomic systems, namely, the molecular dynamics, has been recently boosted due to utilization of graphic processors (GPUs).

Over the years, GPUs have evolved into massively parallel processors that are capable of performing certain types of computations much more efficiently than CPUs. The computational algorithms that are most similar to the tasks performed for rendering 3D graphics have been ported in recent years to GPU architecture, and in many cases the performance of such algorithms has been reported to increase by orders of magnitude in comparison with standard CPU versions. In particular, programs used for studies of macromolecular systems, such as Amber,⁷ NAMD,⁸ and Gromacs,⁹ have been ported to GPUs.^{10–13} The efficiency of the respective GPU versions is so high that the GPU versions of Amber and Gromacs are currently considered the main

production version of the programs. Indeed, in typical biomolecular simulations the performance of the Amber program executed on the single server equipped with a high-end graphics card is higher than the maximal performance that can be obtained on a cluster with hundreds of CPUs.¹²

This big shift toward GPU in MD was not repeated in the Monte Carlo simulations. This is due to the fundamental difference between the two simulation protocols. In MD a new state is generated deterministically by moving all the atoms of the system at once by a very small step, which requires at least $O(N \ln N)$ evaluations of interatomic forces. Contrastingly, a single step of the MC protocol relies on a local random modification of a system. The move acceptance rule (Metropolis criterion) depends only on energy change due to the move, and that ΔE value may be evaluated at $O(\ln N)$ or even at $O(1)$ cost. If energy of the trial configuration is lower than that of the current state, then the move is accepted; otherwise, it is accepted with a probability dependent on the energy difference. In the latter case the move acceptance probability decays as $\exp(-\Delta E/kT)$. Therefore, in practice, only a very small portion of a system (e.g., a single molecule, amino acid side chain, or a group of atoms) is randomly selected and modified. The energy contribution is evaluated

Received: November 18, 2018

Published: March 25, 2019

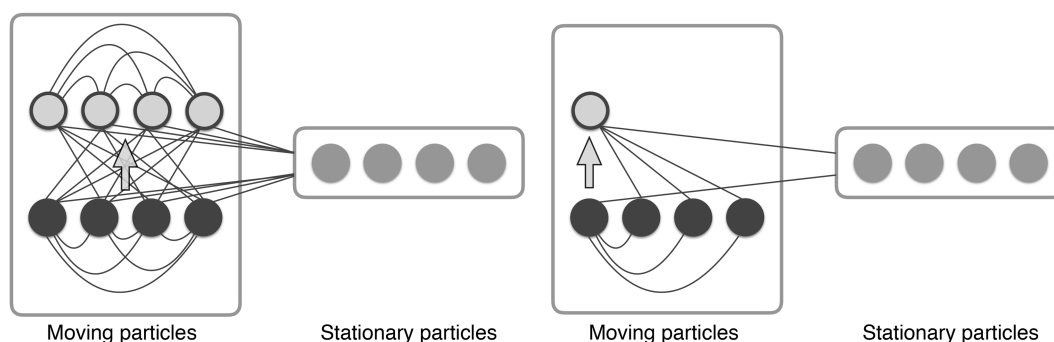


Figure 1. Schematic representation of the interactions in the MultiStep algorithm (left) in comparison with the single step of sequential MC algorithm (right). The system consists of eight particles: four of them are moving, and four are stationary. All interactions within the moving part are shown explicitly, and the interactions with the stationary part are shown as a group. There are 56 pairwise interactions in the left panel. This number is obtained as a sum of 32 interactions between moving and stationary particles and 24 interactions between moving particles. The first number is obtained as $2 \times 4 \times 4$ (two states of moving particle, four moving particles and four stationary particles). The second number is obtained by the following count. For the first particle, we compute 12 interactions: 2 (two states) $\times 3$ (other particles) $\times 2$ (states of other particles). For the second particle we compute 8 interactions $= 2 \times 2 \times 2$. Finally, we compute $2 \times 2 = 4$ interactions between the third and fourth particle. Hence, there are $12 + 8 + 4 = 24$ interactions within the moving part. On the right panel, there are 14 pairwise interactions. To move all particles the MultiStep must be called twice; hence, 112 pairwise interactions are computed. To move all particles in the sequential manner 8 steps are needed. For each step we need to compute 7 pairwise interactions before a move and 7 after. Hence, $8 \times 14 = 112$ pairwise interactions are computed as well.

twice—before and after the move—only for the relevant interactions, i.e., the ones that have been changed due to the move. A single MC move is relatively cheap in terms of computational cost. Yet, a round of trial moves that involves all particles is required to match a single MD step. In contrast to MD, all these moves have to be executed in strictly sequential manner.

In large molecular systems, computation of the energy in the new state consumes more than 99% of the execution time. When the potential energy of a given system consists of two-body interactions, its parallelization is relatively straightforward and leads to great reduction in wall-clock computation time. Nevertheless, the computational effort of the single MC step is too small to give sufficient work to GPU. The GPU architecture is optimized toward parallel execution of thousands of small identical tasks. For a programmer a GPU is more similar to a vector processor, with length of the vector approaching several thousands of units, than to a multicore CPU. Computation of energy between the single particle and the rest of the system is a task too small for splitting into thousands of parallel tasks. A seemingly simple solution of this problem would be to move more particles within a single step. Unfortunately, this solution rapidly decreases efficiency of MC sampling.

In the current article we propose a simulation protocol that overcomes this limitation. In this protocol multiple trial states are generated for a set of moving particles. The final state is identical to the state achieved in the sequential protocol applied for the identical set of particles. The protocol also facilitates sampling of rejected states.¹⁴ In the following sections, first we give the detailed account of the theoretical developments. Then, the results of simulations of the test system performed on GPU and CPU are compared and discussed.

2. MATERIALS AND METHODS

In this contribution, we present two closely related algorithmic developments. The first one is a MultiStep Monte Carlo algorithm—the parallel implementation of the ordinary

sequential MC protocol. Second, we present how to recycle states generated on GPUs in rejected state sampling. A typical MC simulation consists of sweeps, each sweep comprised of elementary MC moves. There are multiple approaches to the generation of these moves that are equivalent to the limit of long trajectory. In particular, one of the possible realizations is random selection of a candidate particle in every move. Another possibility is applying movement to a number of particles in the single sweep,¹⁵ and this method is applied in the current algorithm. In this work we ensure that particles are moved in random order, and the same particle cannot be moved more than once in a given sweep. What is more, the particles included in a sweep are selected randomly. Therefore, only the expected number of steps is identical for all particles. In the limit of a very large number of sweeps, this protocol is equivalent to the standard one. We also assume that energy function consists only of single-body and two-body interactions. With these constraints, the proposed algorithm produces an identical sequence of moves as a sequential protocol, assuming that both are using the same sequence of random numbers. For simplicity we also assume that the elementary step involves perturbation of a single particle. The presented algorithm however is not constrained to such moves. In a general case it is sufficient that nonoverlapping subsets of the system are moved in the sequence of steps.

2.1. MultiStep Algorithm. Let us assume that a system consists of N particles, and M randomly chosen particles will be moved in a sequence of M moves (i.e., during one multistep); hence, $N - M$ of particles will not move (see Figure 1).

We may denote the initial configuration of the i -th particle as X_i^0 and trial configuration as X_i^1 . Without the loss of generality, we may assume that the first M particles could be moved, and the last $N - M$ particles are nonmoving (fixed). Hence, the accessible configuration space of the system can be described as

$$\{X_1^{0 \vee 1}, X_2^{0 \vee 1}, \dots, X_M^{0 \vee 1}\} \times \{X_{M+1}^0, X_{M+2}^0, \dots, X_N^0\} \quad (1)$$

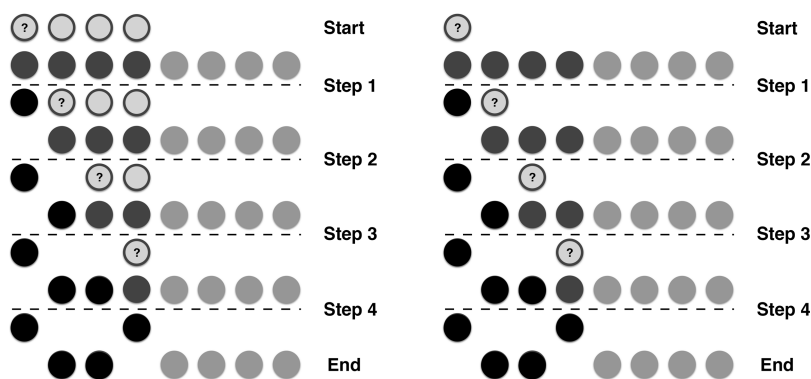


Figure 2. Schematic representation of the parallel MultiStep MC protocol (left) with corresponding traditional sequential simulation (right). The stationary and moving particles are gray and dark gray, respectively. The trial configurations of moving particles are light gray with darker contour. The particles that completed the move are black. The active trial configuration is denoted by a question mark. In this example we assume that the first and last of the four particles have been successfully moved (black circles go up), while moves altering the other two were not accepted. The end configurations are the same in both simulations. In the parallel protocol all the energy contributions are evaluated in a parallel fashion, and only the Metropolis criterion is applied sequentially.

where $X_i^{0\vee 1}$ represents two alternative states of the i -th particle. The pairwise interaction energy of the system can be split into three terms

$$E = \sum_{i=1}^{M-1} \sum_{j=i+1}^M E_{ij} + \sum_{i=M+1}^N \sum_{j=1}^M E_{ij} + \sum_{i=M+1}^{N-1} \sum_{j=i+1}^N E_{ij} \quad (2)$$

where E_{ij} denotes the energy of interaction of the i -th particle with the j -th particle.

The first term is the energy of interactions within the moving part of the system. The second term is the energy of the interactions between the moving and the stationary parts of the system. The third term is not changed by any of the moves, and therefore it may be omitted from further considerations since we are interested only in the difference of the energy between the trial and the initial configuration.

The first term can be written as

$$\begin{aligned} E_{\text{moving}}^{\text{moving}} &= \sum_{i=1}^{M-1} \sum_{j=i+1}^M E(X_i^{0\vee 1}, X_j^{0\vee 1}) \\ &= \sum_{i=1}^{M-1} \sum_{j=i+1}^M \{E(X_i^0, X_j^0) \vee E(X_i^0, X_j^1) \vee E(X_i^1, X_j^0) \vee E(X_i^1, X_j^1)\} \end{aligned} \quad (3)$$

and analogously the second one as

$$\begin{aligned} E_{\text{fixed}}^{\text{moving}} &= \sum_{i=1}^M \sum_{j=M+1}^N E(X_i^{0\vee 1}, X_j^0) \\ &= \sum_{i=1}^{M-1} \sum_{j=i+1}^M \{E(X_i^0, X_j^0) \vee E(X_i^1, X_j^0)\} \end{aligned} \quad (4)$$

In the final state, only one of each of the alternative terms in eqs 3 and 4 contributes to the final energy of the system, depending on the selected state of each moving particle, which is reflected by logical OR operation (marked as $\vee$) in the formulas above. As a comparison, in a standard sequential MC protocol, eq 3 is never used, and eq 4 is reduced to

$$E_{\text{fixed}}^{\text{moving}} = \sum_{j=2}^N E_{1j} = \sum_{j=2}^N E(X_1^{0\vee 1}, X_j^0) \quad (5)$$

Once all energy terms in eqs 3 and 4 are computed in parallel, the algorithm enters the sequential part. For each particle, the following steps are performed. First, the change of the energy of the trial configuration is computed, using appropriate precomputed terms from the particle–particle interaction array. This sum can be parallelized to some extent using the parallel reduction algorithm. Then, the decision whether the move is accepted or not is made. This is a purely sequential part of the algorithm. Finally, the energy of the system is updated using the appropriate terms from the precomputed particle–particle interaction array. This step can also be performed in parallel using the parallel reduction algorithm. The schematic representation of the MultiStep algorithm in comparison with the sequential MC is illustrated in Figure 2. The more detailed description of the single sweep of all particles in the MultiStep algorithm is presented in Listing 1.

Algorithm 1: The single step of the MultiStep MC algorithm

Data: A set $\{X_i^0\}$ of starting configurations of particles in the molecular system S ;
N - a number of particles in the molecular system S ;
M - a number of particles that move in a single step;
Energies of interactions: $E_{ij}^{\alpha\beta}$ of i -th and j -th particles, where $\{\alpha, \beta\} = \{0, 1\}$,
0 denotes a starting position and **1** a trial position. Energies are stored in arrays E^{00} , E^{01} , E^{10} and E^{11} , respectively.
Result: A set of new configurations of particles $\{X_i^1\}$
begin
 Assign randomly M particles in S to *MovingSet*
 Assign $S \setminus \text{MovingSet}$ to *StationarySet*
 /* Parallel section that fully utilises GPU */
 For each particle i in the *MovingSet* generate trial configuration X_i^1
 Compute interactions within *MovingSet* as in Eq. 3 and update appropriate parts of arrays E^{00} , E^{01} , E^{10} and E^{11} .
 Compute interactions between *MovingSet* and *StationarySet* as in Eq. 4 and update appropriate parts of arrays E^{00} and E^{10} .
 Compute energy of each moving particle in starting and trial configurations as:
 $E_i^0 \leftarrow \sum_k E_{ik}^{00}$ and $E_i^1 \leftarrow \sum_k E_{ik}^{01}$
 /* The section that is executed within single block */
for each particle i in *MovingSet* **do**
 In parallel: $r \leftarrow$ random value from uniform distribution on $(0, 1)$
 In parallel: $\text{threshold} \leftarrow -\frac{1}{\beta} \log(r)$
 In parallel: $X_i^f \leftarrow X_i^0$
begin
 /* The critical section performed by a single thread */
 /*
 if $(E_i^1 - E_i^0) < \text{threshold}$ **then**
 Move accepted update final configuration
 $X_i^f \leftarrow X_i^1$
 In parallel: if *Move Accepted* **then**
 Update energy terms in subsequent atoms if $k > i$ **then**
 $E_k^0 \leftarrow E_k^0 - E_{ik}^{00} + E_{ik}^{10}$
 $E_k^1 \leftarrow E_k^0 - E_{ik}^{01} + E_{ik}^{11}$

It is worth noting that the MultiStep method is work-efficient (see Figure 1). In the sequential Monte Carlo protocol, one needs to compute $N_{\text{seq}} = 2M(N - 1)$ energies of pair interactions to move M particles in a system consisting of N particles. In the MultiStep protocol:

$$\begin{aligned} N_{\text{multistep}} &= \frac{1}{2}4M(M - 1) + 2M(N - M) \\ &= 2M(N - M + M - 1) = 2M(N - 1) = N_{\text{seq}} \end{aligned}$$

The first term is from calculating energies of pair interaction in the “moving” part. Each of M “moving” particles is in two states $\{0, 1\}$, and we are counting each pair only once. The second term is from calculating energies of pair interaction between “moving” and “fixed” parts. As before, each of M “moving” particles is in two states, and there are $N - M$ “fixed” particles.

A detailed description of the computations within the algorithm for a very simple system (1D Ising model) is provided in the Supporting Information.

2.2. Implementation of MultiStep on GPU. Both GPU architecture and the CUDA programming model have been described numerous times,^{10,16–19} hence we present only a very brief summary of the most important features, stressing those that influence the design of the current algorithm.

2.2.1. GPU Architecture. GPU resides on the coprocessor card equipped with its own memory. The program utilizing GPU is run on CPU and executes computational kernels on GPU. The data utilized by kernels have to be transferred to the GPU card over a relatively slow and high-latency PCIe link, and hence the efficient programs minimize communication between CPU and GPU.

GPU is a multicore processor; usually the number of cores in high-end chips, which are used for computations, varies between 12 and 16. The core of the GPU is called streaming multiprocessor (SM). Each SM has a hardware support for parallel execution of multiple threads. It has multiple execution units specialized in different tasks. For example, one SM in the GK210 GPU in Tesla K80 cards has 192 and 64 adders/multipliers, for floating point and double precision, respectively; 160 units for integer add or logical operations; and 32 units for integer shifts or multiplications, etc. One can notice that all these numbers are multiples of 32, which is the number of threads that constitutes the so-called warp. All threads in a warp execute exactly the same machine instructions in a lock step in the so-called single instruction multiple threads (SIMT) model. Each thread has its own private registers, but starting from Kepler generation of CUDA, GPU threads within a single warp can directly read contents of the other's registers using *shuffle* instructions. Multiple warps (up to 32) are grouped in a block of threads. All threads of a block reside on a single multiprocessor and share its resources, in particular shared memory and register file. Threads within a single block can be synchronized using the `__syncthreads()` function. On recent architectures a multiprocessor can execute up to 2048 threads in parallel. The highest level of thread hierarchy is called a grid. It is a collection of blocks that can be executed in any order, on any available multiprocessor. All threads in a grid execute the same kernel function, but generally there is no way to synchronize threads across blocks within a single kernel. Hence, the synchronization on this level is performed by issuing multiple kernels. The GPU is capable of executing tens of thousands of threads in parallel. What is more, such

parallelism is required to achieve good performance of the GPU.

2.2.2. Details of Implementation of the MultiStep Method on GPU. Let us review the main steps of the MultiStep algorithm in the context of the requirements of GPU architecture.

A single multistep consists of four obligatory and one optional steps:

1. Initialization: This step includes generation of random numbers, generation of new configurations for particle and some additional auxiliary operations. This is a moderately parallel step, which does not involve thousands of threads.
2. Compute energy of pairwise interactions between “fixed” and “moving” parts. This step is highly parallel, and it can fully utilize all available GPU resources.
3. Compute energy of pairwise interaction inside the “moving” part. The degree of parallelism of this step depends on the number of moving particles. When the number of moving particles is sufficiently high, it can fully utilize GPU.
4. MultiStep algorithm. This is a sequential operation that needs to be performed within a single block.
5. Sampling of rejected states (optional). The level of parallelism depends on the number and size of coarse-grained steps.

The most computationally demanding are steps two and three that involve computation of energy between moving molecules and all other molecules. These computations are performed in single precision, that is, more efficient on GPU than CPU. For example, on K210 GPU the floating point performance in single precision (SP) is triple the performance in double precision (DP). The difference is even higher on other GPUs—on the newer Maxwell architecture the performance in SP is 32 times higher than in DP. What is more, computation of electrostatic interaction involves computing the inverse square root. GPU has an intrinsic operation for the inverse square root in SP, which is executed as a single instruction. There is no such operation for DP computations. Nevertheless, the accumulation of results is performed in double precision variables. This setup is used mostly for debugging purposes—the Monte Carlo protocol is not sensitive to small rounding errors that may arise due to lower precision of computations; hence, all computations could be executed in single precision without influence on the results. However, the floating point operations are sensitive on the order of execution, and therefore the energies may differ between CPU version and GPU one. On GPU one has very limited control of the true order of execution of parallel operations, hence the differences between sequential and parallel algorithms are inevitable. Therefore, the double precision is used for accumulation of energies, to reduce these differences and allow comparisons and debugging.

Once the pairwise energies are computed, the algorithm enters the MultiStep procedure (see 1). In this procedure the emulation of the sequential algorithm is performed. To this end, the aggregation of energy of interactions of each particle with all other particles is performed for both starting and trial states. The energies of interaction of the i -th particle with particles $\{i + 1, \dots, M\}$ can be safely computed in parallel since in this case only the terms for starting positions of atoms with higher index are used. On the other hand, the decision of

which energy terms should be included as the energy of interaction of the i -th particle with particles $\{1, \dots, i-1\}$ can be made only after the state of these particles is known. Therefore, this part of the algorithm is strictly sequential. Hence, the implementation of the MultiStep procedure requires a critical section. On the other hand, once the decision for the i -th particle is made, the result can be communicated to all other particles in parallel.

Since there is a critical section, we need to perform a synchronization of GPU threads. This implies that MultiStep can only be run using one block of threads, as there is no safe way to perform an efficient synchronization of threads in many CUDA blocks. The size of the block is equal to the number of “moving” particles.

The critical section is the slowest part of the entire algorithm since only one thread is executing. Therefore, reducing the number of operations in this section is crucial. To this end we have rewritten the Metropolis criterion using following relation

$$u < \exp(-\beta \cdot \Delta E) \leftrightarrow -\frac{1}{\beta} \log(u) > \Delta E \quad (6)$$

where u is a standard uniform random number.

When the second form of the criterion is used, each thread can generate a random number in parallel, compute the logarithm, multiply the value by $-1/\beta$, and store the output in a local register, which can later be used in a critical section as a threshold value. This approach reduces the critical section to only one comparison and one store to the shared memory if the move was accepted.

One additional optimization takes advantage of the block-level parallelism. During the MultiStep kernel the GPU is underutilized because it is limited to only one block. To increase the utilization we perform the initialization for the next step in parallel with this step, by generation of random translations and random rotations for subsequent MC moves.

2.3. MultiStep Algorithm with Sampling of Rejected States. The MultiStep algorithm introduced in the previous section is an efficient method for implementation of the sequential algorithm on the massively parallel processor. The intermediary step of the algorithm, namely, the computation of the partial energy tables, opens the opportunity to increase the efficiency of sampling of the energy and other observables that can be cheaply collected together with the energy. To this end, one can use a method proposed by Frenkel.¹⁴ It increases the sampling efficiency of the MC scheme by also including the averages of the rejected states along the trajectory. In particular, Frenkel proposed that when a single “coarse-grained” MC simulation step is constructed as the sequence of fine-grained “microsteps” the contribution of all possible microsteps to the single coarse-grained step can be included in the averages. The parallel generation of M trial moves of M particles leads to 2^M distinct states. The coarse-grained move corresponds to selection of one of these states as a final state. In the standard MC procedure only M states along the selected transition path contribute to the averages. In the parallel implementation one can in principle generate all 2^M alternative microstates; however, this may be impossible in practice for large M . Even for relatively small $M = 16$, the number of microstates is 65 536; for $M = 32$, the number of possible microstates is larger than 4 billion; and for higher M , it quickly becomes astronomical and practically untractable. Fortunately, the selection of the coarse-grained steps is not constrained to the end points of the multistep. One may formally split the

multistep into several independent coarse-grained steps, with their size adjusted to computational efficiency. Assuming that the parallel multistep of size M is split into k coarse-grained steps, the number of states included in the averages is equal to $k \cdot 2^{M/k}$. For example, for $M = 256$ and $k = 32$ the number of included microstates is 8192, which is 32 times more than in the traditional sampling.

Following the eq 2.2 in ref 14, we compute the averages of the observables in each coarse-grained step as

$$\langle A \rangle = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{k \leftrightarrow i} P_B(k) A(K)}{\sum_{k \leftrightarrow i} P_B(k)} \quad (7)$$

The selection of the final coarse-grained state can be performed either using the simulated sequential step as in the MultiStep algorithm or by symmetric rule:

$$p(k) = \frac{P_B(k)}{\sum_{k \leftrightarrow i} P_B(k)} \quad (8)$$

Both algorithms generate proper sampling of the configuration space, despite that the probability of transition between the initial coarse-grained state

$$\{X_1^0, \dots, X_M^0; X_{M+1}^0, \dots, X_N^0\}$$

and a final coarse-grained state

$$\{X_1^{\alpha_1}, \dots, X_M^{\alpha_M}; X_{M+1}^0, \dots, X_N^0\}$$

is different for these protocols. The symmetric rule imposes a stronger balance condition. Nevertheless, since the probability of transition between all microstates in the ensemble is identical to the ensemble average over multiple Metropolis trajectories, both transition rules can be used. Consequently, the selection of either transition rule is irrelevant for the long-term convergence of the averages and can be based only on the efficiency of implementation.

One should note that the approach proposed here differs in a small but important detail from the original Frenkel proposal. He proposed that the elementary moves should be performed by noninteracting particles, whereas the less stringent condition is used here—it is sufficient that nonoverlapping parts of the system are moved in parallel.

2.4. Implementation of Frenkel Sampling. In the current implementation the coarse-grained move corresponds to the move of eight particles that results in 256 different microstates. The state of each particle is a binary switch indicating whether a particle has moved (state 1) or not (state 0). The states are ordered in such a way that each subsequent state differs by a state of a single particle. To this end, the Gray codes²⁰ are used. Hence, the difference of energy between two subsequent states can be obtained as the difference of energy of two states of a single particle. Therefore, the computation of the energy difference between the initial state 0 $E_{\text{Gray}[0]}$ and i -th state $E_{\text{Gray}[i]}$ can be obtained in two steps. In the first step, the energy differences between subsequent states are computed. In the block of 256 threads each thread computes the difference of energy between two states $E_{\text{Gray}[i]} - E_{\text{Gray}[i-1]}$. The following table is obtained:

$$(0, E_{\text{Gray}[1]} - E_{\text{Gray}[0]}, E_{\text{Gray}[2]} - E_{\text{Gray}[1]}, \dots, E_{\text{Gray}[255]} - E_{\text{Gray}[254]}) \quad (9)$$

Then we obtain the prefix sum and we get the following table:

Table 1. Comparison of Mean Energies and Mean Densities of 512 TIP5P Water Molecules As a Function of Pressure at 25 °C

p [atm]	results from ref 21, Table 7a		GPU MultiStep	
	$-E$ [kcal/mol]	density [g/cm ³]	$-E$ [kcal/mol]	density [g/cm ³]
1	9.867 ± 0.006	0.9991 ± 0.0014	9.896 ± 0.007	1.0042 ± 0.0014
1000	9.929 ± 0.005	1.0528 ± 0.0007	9.951 ± 0.007	1.0533 ± 0.0010
2000	9.968 ± 0.004	1.0922 ± 0.0007	9.992 ± 0.007	1.0934 ± 0.0010
3000	9.994 ± 0.015	1.1276 ± 0.0009	10.024 ± 0.006	1.1270 ± 0.0011
4000	10.04 ± 0.012	1.1591 ± 0.0011	10.052 ± 0.007	1.1571 ± 0.0007
5000	10.078 ± 0.001	1.1828 ± 0.0011	10.079 ± 0.005	1.1831 ± 0.0006
6000	10.060 ± 0.015	1.2051 ± 0.0009	10.104 ± 0.005	1.2073 ± 0.0008
8000	10.105 ± 0.013	1.2453 ± 0.0016	10.141 ± 0.007	1.2489 ± 0.0009
10000	10.156 ± 0.010	1.2830 ± 0.0007	10.176 ± 0.007	1.2847 ± 0.0008

$$(0, E_{\text{Gray}[1]} - E_{\text{Gray}[0]}, E_{\text{Gray}[2]} - E_{\text{Gray}[0]}, \dots, E_{\text{Gray}[255]} - E_{\text{Gray}[0]}) \quad (10)$$

Once the energy difference from the state $E_{\text{Gray}[0]}$ is obtained, we can compute the normalizing factor for each block of 256 values:

$$Z = \sum_{i=1}^{i=255} \exp(-\beta(E_{\text{gray}[i]} - E_{\text{gray}[0]})) \quad (11)$$

Finally, we can update the histogram for the energies. The weight associated with each state is given by

$$h_i = \frac{1}{Z} \exp(-\beta(E_{\text{gray}[i]} - E_{\text{gray}[0]})) \quad (12)$$

One should note that the state 0 is not included in the sum, and the histogram bin corresponding to this state is not updated since it is the final microstate of the previous coarse-grained step and therefore has already been counted.

2.5. Test System. The proposed algorithms were implemented and applied on CPU and GPU for the TIP5P water model.²¹ CPU implementation was a single-threaded sequential algorithm executed on a single core. It was used as a reference, both for checking the correctness and efficiency of the GPU version. In particular, we examined whether the simulations performed with the MultiStep algorithm on GPU were equivalent to simulations performed with the sequential algorithm. The simulation conditions were identical to those in the original publication. The computations were performed in an NPT ensemble for 512 water molecules contained in a cubic box with periodic boundary conditions. The cutoff for water–water interactions was 0.9 nm. The test system is relatively small—in the standard conditions around one-fifth of the entire system (roughly 100 molecules) is within the cutoff range of every molecule. Therefore, the system cannot be efficiently divided into many parallel Monte Carlo simulations using domain decomposition. TIP5P water is an example of the computationally intensive system. This is because for each pair of molecules within the cutoff range one has to calculate one Lennard-Jones potential (interaction between oxygen atoms) and 16 contributions to the electromagnetic potential for charged sites. Hence, it is ideal to show the merit of the algorithm that relies on parallelizing computations of interaction potential.

The goal of the tests was to measure efficiency of the proposed algorithm on the real system and not exact repetition of the original Mahoney and Jorgensen study; hence, only a subset of simulations reported in the original paper were

carried out. For each pressure, we performed 20 independent MC simulations (with different seeds for PRNGs); each simulation consisted of 1000 million MC steps; and averages were collected for the last 300 million steps. Volume changes were attempted every 1920 steps, and overall acceptance rates for translations and rotations were around 40%.

3. RESULTS AND DISCUSSION

To conduct our studies, we have developed three separate programs that perform the very same simulation: (i) the reference sequential implementation of the MCMC algorithm operating on CPU, (ii) a sequential implementation of MultiStep algorithm operating on CPU, and (iii) a parallel GPU implementation of MultiStep algorithm. The full set of simulations presented in this contribution is computationally demanding. Therefore, it was performed with program (iii) only (the parallel MultiStep algorithm implemented on GPU). However, to examine the correctness of the proposed method, several trial runs were performed in various conditions using all three programs using the same random seed. The two CPU implementations (ii and i) gave identical results, provided that the simulations were started from the same random seed. On the other hand, trajectories produced by the program (iii) were initially identical but slowly diverging. This was caused by a different order of summation on GPU which leads to different numeric round-off errors in energy which in turns causes different Metropolis criterion decisions. Nevertheless, observables computed by program (iii) were physically identical to those calculated with program (ii).

3.1. Energy and Density of TIP5P Water. We have compared mean energies and densities from our MultiStep GPU simulation with the ones reported by Mahoney et al.²¹ Results are shown in Table 1 and Figures 3 and 4. The density of water in different pressures is reproduced very well. The

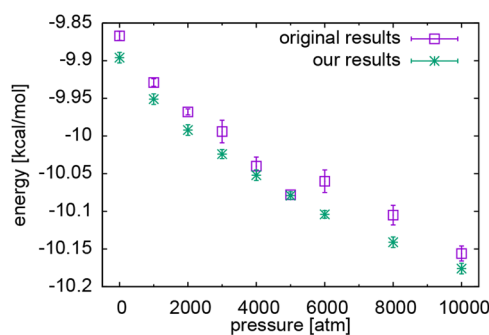


Figure 3. Energy of TIP5P water as a function of pressure at 25 °C.

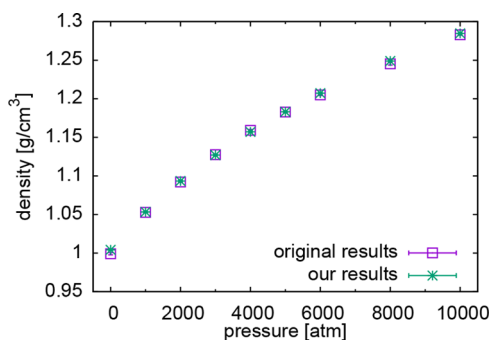


Figure 4. Density of TIPSP water as a function of pressure at 25 °C.

differences between simulations are within standard deviations. The small but systematic deviation of 0.024 kcal/mol between energies reported in the original work and in the current study is within variation of results obtained in the individual runs of the current study. Nevertheless, this deviation is systematic: the p -value of the hypothesis, that all differences result from random fluctuation, is 0.0005.

The difference is most likely due to the difference in the protocol used. The original results were obtained in a single simulation for each data point, and total simulation length at each pressure varied between 60 and 300 million steps. The simulation error was estimated using the variance in the single simulation run. In the current study, the equilibration phase was 700 million steps and data collection 300 million steps. What is more, at each pressure 20 runs were carried out, and the reported results are obtained as average and standard error from 20 independent simulations. The observation of the simulations suggests that the simulation times in the original study were too short for the proper equilibration; hence, the results were biased to higher energies, and the reported simulation errors were underestimated.

The energy curve obtained in the current study is smooth, whereas the original curve varies wildly for pressures between 4000 and 6000 atm. One should note that values obtained for 4000 and 5000 atm in the original study agree very well with the current results. This is probably due to fortuitous selection of the initial configuration that resulted in quick equilibration to the low-energy state that was close to the true equilibrium. Another factor that may contribute is high sensitivity of the system to even slight changes in physical constants and parameters used in simulation. The slight changes due to different rounding of physical constants can lead to substantial deviations for the energy of the system, which might contribute to the difference in mean energies. Interestingly, the energy of the system obtained in the current study for 1 atm and 25 °C is closer to the experimental value of -9.92 kcal/mol than results reported in the original study.

3.2. Performance. Performance tests were carried out using five different systems hosting four types of GPU cards. The summary of test hosts is given in Table 2. The code was compiled using nvcc version 8.0 on linux. The CPU version was optimized and compiled using two different compilers: C/C++ GNU (version 4.9.4), as well as Intel C/C++ compiler (version 2015.3.187). While the CPUs installed in the test hosts differ, their performance has minuscule effects on the performance of the GPU version of the simulation code since more than 99% of the time is spent in the GPU performing simulations.

Table 2. Test Systems

system and GPU card	GPU type	CPU type	CPU clock
Titan Black	GK110	Intel i7-4820K 4-core	3.7 Ghz
Tesla K80	GK210	Intel Xeon E5-2650 8-core	2.6 Ghz
GTX 980 Ti	GM200	Intel Xeon E5620 4-core	2.4 Ghz
Titan X	GP100	Intel i7-6850K CPU 6-core	3.6 GHz
CPU	none	Intel Xeon E5-2697v3 14-core	2.1 GHz

The execution time for 512 000 000 steps was 24 490 s for the binary version compiled with the GNU compiler and 6246 s for the code compiled with Intel compiler. The performance of the GPU code was measured for different sizes of the multistep starting from 32 up to 512. One should note that the last value is the size of the entire system; hence, all the atoms are moving, and the kernel for computation of the interactions between moving and fixed atoms is not called.

The execution times and corresponding speedup for the GPU code are presented in Table 3.

It is clear that the small MultiStep sizes are too small for efficient parallelization. The performance of the algorithm is low for MultiStep size 32. It quickly increases when MultiStep size increases and more than doubles with the size equal to 128. The performance is nearly constant for all sizes between 160 and 480. A slight, but consistent, drop of performance is observed when the entire system is moved at once. The maximal performance is achieved for MultiStep size 224 for both Kepler GPUs and for Pascal GPU. The optimal size for Maxwell GPU is 288.

It is interesting to see how the execution time of different kernels changes with the increasing size of the MultiStep. To this end, a profiling of the code was performed on the GM200 GPU. To obtain sufficient statistics the simulation consisting of 5 120 000 MC steps was performed under the control of the nvprof profiler. The total times spent in the six most important kernels are presented in Table 4.

The first four kernels are parallel and, depending on the number of steps in the Multistep, can be highly parallel. The calculateEnergyKernel computes the energy of the entire system for the volume change moves as well as for generation of the time averages. The execution of this kernel does not depend on the size of the Multistep, and it is very easily parallelized. Its contribution to the total time depends on the frequency of volume change moves.

The next two kernels are the most computationally intensive. They compute interactions between particles during ordinary moves. The movingMovingKernel computes interactions of all moving particles, and the movingFixedKernel computes interactions between moving and stationary. In the movingFixedKernel, each block computes interactions of all moving particles with 8 stationary particles, and a single thread is assigned to each moving particle. In the case of movingMovingKernel, each block computes interactions using tiles of the size 8×8 . The efficiency of these kernels depends on the number of moving particles. When the number of moving atoms is small, the number of blocks is small resulting in the underutilized GPU. The performance of the parallel part improves rapidly with increasing size of the multistep. The total time for computing interactions is 1740 ms for MultiStep size 32, and it decreases to 580 and 550 ms, for sizes 256 and 512, respectively. Interestingly, the best performance of the movingMovingKernel is obtained with 25%

Table 3. Performance of the Code for Various Configuration of the Kernels^a

size	execution time				speedup							
					GK110		GK210		GM200		GP100	
	GK1	GK2	GM	GP	gcc	I	gcc	I	gcc	I	gcc	I
32	1035	1180	948	527	24	6	21	5	26	7	46	12
64	709	800	614	351	34	9	31	8	40	10	70	18
96	560	620	469	278	44	11	40	10	52	13	88	22
128	487	553	410	244	50	13	44	11	60	15	100	26
160	451	535	366	227	55	14	46	12	67	17	108	28
192	432	508	349	217	57	14	48	12	70	18	113	29
224	424	483	343	207	58	15	51	13	71	18	118	30
256	448	513	336	216	55	14	48	12	73	19	113	29
288	431	505	330	210	57	14	48	12	74	19	117	30
320	434	518	351	210	56	14	47	12	70	18	117	30
352	448	529	349	232	55	14	46	12	70	18	106	27
384	447	526	352	232	55	14	47	12	70	18	106	27
416	454	534	344	233	54	14	46	12	71	18	105	27
448	437	509	344	232	56	14	48	12	71	18	106	27
480	430	505	344	236	57	14	48	12	71	18	104	26
512	473	546	368	249	52	13	45	11	66	17	98	25

^aTime in seconds for execution of 512×10^6 MC steps. Speedup in comparison with a standard sequential single core version of MC algorithm compiled with gcc and Intel (denoted as I) compilers is shown. GK1 and GK2 denote Titan Black and Tesla K80. The GM200 denotes GTX 980 Ti, and GP100 denotes Titan X. The best results for each GPU are highlighted in boldface.

Table 4. Split of the Computation Time between Various Kernels for Different Sizes of PMSMC Algorithm^a

MultiStep size	execution time (ms)					fraction of total time (%)				
	32	64	128	256	512	32	64	128	256	512
calculateEnergyKernel	92	97	98	99	109	2.4	3.9	5.1	5.4	4.8
movingMovingKernel	538	307	272	331	552	13.9	12.5	14.3	18.0	24.5
movingFixedKernel	1202	636	430	250	0	31.1	25.8	22.6	13.6	0.0
prepareMultiStepKernel	268	199	127	198	285	6.9	8.1	6.7	10.7	12.7
prepareRandomKernel	670	340	187	105	83	17.3	13.8	9.8	5.7	3.7
multistepKernel	835	696	664	710	1025	21.6	28.3	34.9	38.6	45.6
other auxiliary kernels	263	190	123	149	197	6.8	7.7	6.5	8.1	8.7
total	3868	2465	1902	1841	2251	100.0				

^a5 120 000 steps of the algorithm executed on the GP100 GPU. Only the GPU time is shown.

of the maximal number of threads that can reside on the single multiprocessor of the GPU. Increasing the size of the tiles to 16×16 did not result in better performance even for large MultiStep sizes.

The prepareMultiStepKernel sets the stage for the serial MultiStep by precomputing energy difference between two states of all moving particles. The initial state is identical for all particles, and the final state differs by movement of the single particle. The amount of work for a single call is proportional to the square of MultiStep size, while the number of calls is inversely proportional to the MultiStep size. For small MultiStep sizes, the insufficient parallelism of the task decreases the overall performance, while for larger ones the increased amount of work takes over. The optimal MultiStep size for this kernel is 128.

The tasks performed by the prepareRandomKernel have insufficient parallelism for full utilization of the GPU. This kernel splits the system into moving and stationary parts and prepares the random moves for the moving particles. It is called once per MultiStep. The execution parameters for this kernel were selected to give optimal performance for MultiStep size equal to 256. It limits the performance for small MultiStep sizes, but for larger sizes its significance decreases quickly.

The multistepKernel performs two tasks. The main task is a propagation of the simulation using precomputed energies of interactions between particles. The auxiliary task is generation of moves for the next step. The main task has to be performed by a single block of threads and consists of two steps. The first one is the decision whether to accept the move of a single molecule. It is performed within a serial single-threaded section that is a main bottleneck of the entire algorithm. In the second step, the initial energy for each subsequent move is modified to reflect the consequences of the decision. This step is parallel, but the parallelism is limited to a single block. Despite this, the algorithm minimizes the sequential part; the multistepKernel takes roughly 40% of the computation time. The parallel section has a variable contribution depending on the number of steps in MultiStep. For the small number of steps the parallel section has limited significance, while for the large step the amount of work becomes substantial and starts to influence the overall performance.

The best performance, i.e., 30-fold speedup, in comparison with single threaded performance on CPU was obtained for the Titan X card based on the GP100 GPU; however, even the older GPUs compared quite favorably with the CPU versions. Interestingly, the CPU performance depends to a very large

extent on the compiler used to produce the binary code. The Intel compiler generated the code that runs nearly four times faster on the same hardware. Despite significant effort that was put into optimizing the code and compilation procedure, we were unable to close the gap between the Intel and gcc compiler. Therefore, the speedup in comparison with the code compiled with gcc was nearly 120-fold.

3.3. Sampling of the Rejected States. The procedure for sampling of the rejected states was applied for the trajectories used to validate physical correctness of the simulation protocol. Its application did not introduce any significant changes to the results of simulations, despite 32-fold increase of the number of samples included in the averages. This is caused by the presence of large and slowly decaying fluctuations of the total energy of the system. The modifications of energy due to moves of eight particles have too small scale in comparison with the large-scale reorganizations of the system. The application of the Frenkel sampling within the MultiStep protocol comes with a performance penalty. The procedure is executed within the parallel section of the MultiStep kernel, adding several costly operations to the procedure that itself is a bottleneck of the algorithm. This overhead adds roughly 50% of computational time to the simulation in comparison with the standard version.

3.4. Discussion. The MultiStep MC algorithm is a general procedure that is designed for Markov Chain Monte Carlo simulations. It is particularly useful when very long simulations are required to study given phenomena. Here we have shown that a useful range of a single simulation can be extended by a factor of 30 in comparison with a sequential implementation of the algorithm using the state of the art CPU and compiler. The performance improvement over the CPU code compiled with gcc compiler was more than 100-fold. The relatively small test system used limited the achievable speedup considerably since it consisted of only 512 particles. The bottleneck of the simulation protocol is the sequential MultiStep procedure, as it is performed by a single thread within the single block. The larger system can be split into several noninteracting cells, and the MultiStep procedure can be applied in parallel for each of them. In such a case, multiple sequential sections of the algorithm performing the MultiStep procedure on different noninteracting fragments could be performed in parallel using more blocks. In this way, the fraction of time spent in the sequential part could be reduced from roughly 45% in the current system to less than 5%, leading to nearly doubling the performance of the algorithm relative to the CPU version.

The Markov Chain Monte Carlo simulations are often performed using multiple copies of the system. For example, in the replica exchange protocol,^{22,23} which is for instance used in protein folding studies,²⁴ several copies of the same molecule are simulated in the different temperatures in parallel. In the case of simulated annealing,²⁵ hundreds of thousands of jobs are executed at the same time, e.g., on BOINC platform. It is a relatively easy task to combine runs from multiple copies into a single simulation and thus reduce the influence of the sequential part to minimum. Nevertheless, even for the current small system, the performance of the MultiStep algorithm executed on a single GPU card, without the easily implemented improvements mentioned above, is on par with that, which is achievable for sequential CPU code on the high end dual socket multicore server, with the code compiled using the commercial compilers.

The MultiStep Markov Chain Monte Carlo algorithm introduced in the current study is not limited to the GPU. The procedure can also be implemented on the multicore CPUs. However, it is feasible only when the goal of the study is to achieve the longest possible single trajectory of the system under scrutiny within a given time or to perform a prescribed number of steps in the shortest possible wall time. The possible applications include, for example, simulations of the protein folding with atomistic resolution or atomistic simulations of diffusive processes in material science.

■ ASSOCIATED CONTENT

Supporting Information

The Supporting Information is available free of charge on the ACS Publications website at DOI: [10.1021/acs.jctc.8b01168](https://doi.org/10.1021/acs.jctc.8b01168).

Detailed description of the computations within the algorithm for a very simple system (1D Ising model) is provided for better visualization of the algorithm (PDF)

■ AUTHOR INFORMATION

Corresponding Authors

*E-mail: dgront@chem.uw.edu.pl.

*E-mail: w.rudnicki@uw.edu.pl. Phone: +48 85 738 8276.

ORCID

Witold R. Rudnicki: [0000-0002-7928-4944](https://orcid.org/0000-0002-7928-4944)

Funding

Computations were performed at the Computational Centre of the University of Białystok, grant GO-019. Research was partially funded by National Science Centre, Poland, grant to D.G. no. 2018/29/B/ST6/01989.

Notes

The authors declare no competing financial interest.

■ REFERENCES

- (1) Ulam, S.; Richtmyer, R.; Von Neumann, J. Statistical methods in neutron diffusion. *LAMS-551*; Los Alamos National Laboratory, 1947; pp 1–22.
- (2) Metropolis, N.; Ulam, S. The monte carlo method. *J. Am. Stat. Assoc.* **1949**, *44*, 335–341.
- (3) Metropolis, N.; Rosenbluth, A. W.; Rosenbluth, M. N.; Teller, A. H.; Teller, E. Equation of state calculations by fast computing machines. *J. Chem. Phys.* **1953**, *21*, 1087–1092.
- (4) Landau, D. P.; Binder, K. *A Guide to Monte Carlo Simulations in Statistical Physics*, 3rd ed.; Cambridge University Press, 2009.
- (5) Kmiecik, S.; Gront, D.; Kolinski, M.; Wieteska, L.; Dawid, A. E.; Kolinski, A. Coarse-Grained Protein Models and Their Applications. *Chem. Rev.* **2016**, *116*, 7898–7936.
- (6) Christen, M.; van Gunsteren, W. F. F. On searching in, sampling of, and dynamically moving through conformational space of biomolecular systems: A review. *J. Comput. Chem.* **2008**, *29*, 157–166.
- (7) Salomon-Ferrer, R.; Case, D. A.; Walker, R. C. An overview of the Amber biomolecular simulation package. *Wires. Comput. Mol. Sci.* **2013**, *3*, 198–210.
- (8) Phillips, J. C.; Braun, R.; Wang, W.; Gumbart, J.; Tajkhorshid, E.; Villa, E.; Chipot, C.; Skeel, R. D.; Kale, L.; Schulten, K. Scalable molecular dynamics with NAMD. *J. Comput. Chem.* **2005**, *26*, 1781–1802.
- (9) Pronk, S.; Páll, S.; Schulz, R.; Larsson, P.; Bjelkmar, P.; Apostolov, R.; Shirts, M. R.; Smith, J. C.; Kasson, P. M.; van der Spoel, D.; Hess, B.; Lindahl, E. GROMACS 4.5: a high-throughput and highly parallel open source molecular simulation toolkit. *Bioinformatics* **2013**, *29*, 845–854.
- (10) Stone, J. E.; Phillips, J. C.; Freddolino, P. L.; Hardy, D. J.; Trabuco, L. G.; Schulten, K. Accelerating molecular modeling

applications with graphics processors. *J. Comput. Chem.* **2007**, *28*, 2618–2640.

(11) Salomon-Ferrer, R.; Goetz, A. W.; Poole, D.; Le Grand, S.; Walker, R. C. Routine microsecond molecular dynamics simulations with AMBER on GPUs. 2. Explicit solvent particle mesh Ewald. *J. Chem. Theory Comput.* **2013**, *9*, 3878–3888.

(12) Goetz, A. W.; Williamson, M. J.; Xu, D.; Poole, D.; Le Grand, S.; Walker, R. C. Routine microsecond molecular dynamics simulations with AMBER on GPUs. 1. Generalized born. *J. Chem. Theory Comput.* **2012**, *8*, 1542–1555.

(13) Eastman, P.; Friedrichs, M. S.; Chodera, J. D.; Radmer, R. J.; Bruns, C. M.; Ku, J. P.; Beauchamp, K. A.; Lane, T. J.; Wang, L.-P.; Shukla, D.; Tye, T.; Houston, M.; Stich, T.; Klein, C.; Shirts, M. R.; Pande, V. S. OpenMM 4: a reusable, extensible, hardware independent library for high performance molecular simulation. *J. Chem. Theory Comput.* **2013**, *9*, 461–469.

(14) Frenkel, D. Speed-up of Monte Carlo simulations by sampling of rejected states. *Proc. Natl. Acad. Sci. U. S. A.* **2004**, *101*, 17571–17575.

(15) Leach, A. R. *Molecular modelling: principles and applications*; Pearson Education, 2001; p 417.

(16) Lindholm, E.; Nickolls, J.; Oberman, S.; Montrym, J. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE micro* **2008**, *28*, 39–55.

(17) Nickolls, J.; Buck, I.; Garland, M.; Skadron, K. Scalable parallel programming with CUDA. *Queue* **2008**, *6*, 40–53.

(18) Stone, J. E.; Hardy, D. J.; Ufimtsev, I. S.; Schulten, K. GPU-accelerated molecular modeling coming of age. *J. Mol. Graphics Modell.* **2010**, *29*, 116–125.

(19) NVIDIA, *NVIDIAs next generation CUDA compute architecture*; Kepler GK110; 2012.

(20) Gray, F. Pulse code communication. US Patent US2,632,058/1953.

(21) Mahoney, M. W.; Jorgensen, W. L. A five-site model for liquid water and the reproduction of the density anomaly by rigid, nonpolarizable potential functions. *J. Chem. Phys.* **2000**, *112*, 8910–8922.

(22) Swendsen, R. H.; Wang, J. S. Replica Monte Carlo Simulation of Spin-Glasses. *Phys. Rev. Lett.* **1986**, *57*, 2607–2609.

(23) Gront, D.; Kolinski, A.; Skolnick, J. Comparison of three Monte Carlo conformational search strategies for a proteinlike homopolymer model: Folding thermodynamics and identification of low-energy structures. *J. Chem. Phys.* **2000**, *113*, 5065–5071.

(24) Hansmann, U. H. E. Parallel Tempering Algorithm for Conformational Studies of Biological Molecules. *Chem. Phys. Lett.* **1997**, *281*, 140–150.

(25) Kirkpatrick, S.; Gelatt, C. D.; Vecchi, M. P. Optimization by simulated annealing. *Science* **1983**, *220*, 671–680.